

Fundamentals of
PYTHON
DATA STRUCTURES

2ND EDITION

Kenneth A. Lambert





National Economics University
Faculty of Mathematical Economics

EP03.TITH1121 - DATA STRUCTURES AND ALGORITHMS: Trees

- ❑ Instructor: Duc Minh Vu (MFE)
- ❑ Email: minhvd [at] neu.edu.vn



Learning Objectives

1. Describe the features of a tree
2. Describe various types of tree traversals
3. Recognize three common applications where it is appropriate to use a tree
4. Describe the features of a binary search tree and the operations on it
5. Recognize the three common applications where it is appropriate to use a binary search tree
6. Describe the features of an expression tree and the operations on it



Some applications

- ❑ Translating infix expressions to postfix form and evaluating postfix expressions.
- ❑ CPU access—Processes are queued for access to a shared CPU.
- ❑ Disk access—Processes are queued for access to a shared secondary storage device.
- ❑ Printer access—Print jobs are queued for access to a shared laser printer.



Overview of trees

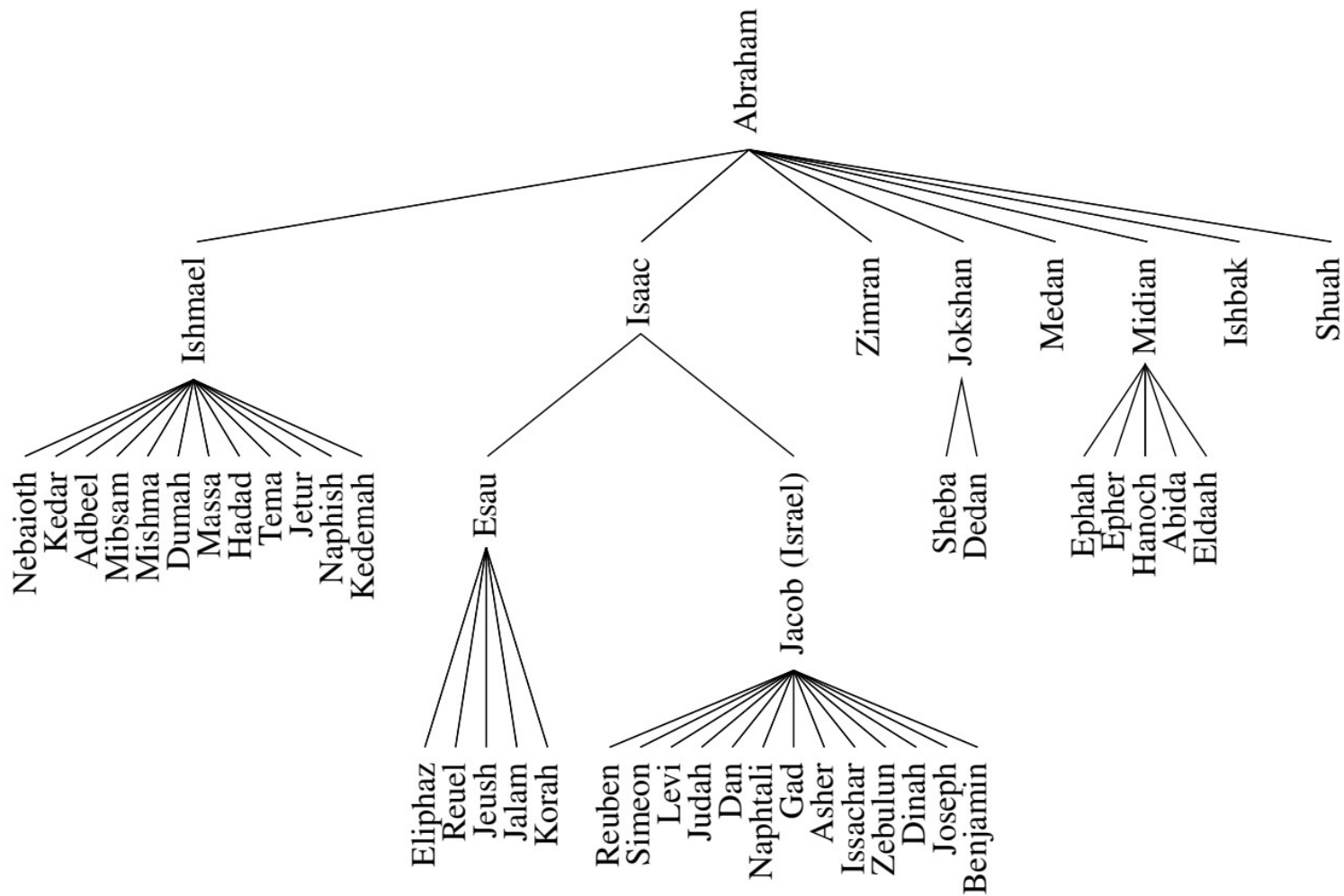


Figure 8.1: A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.



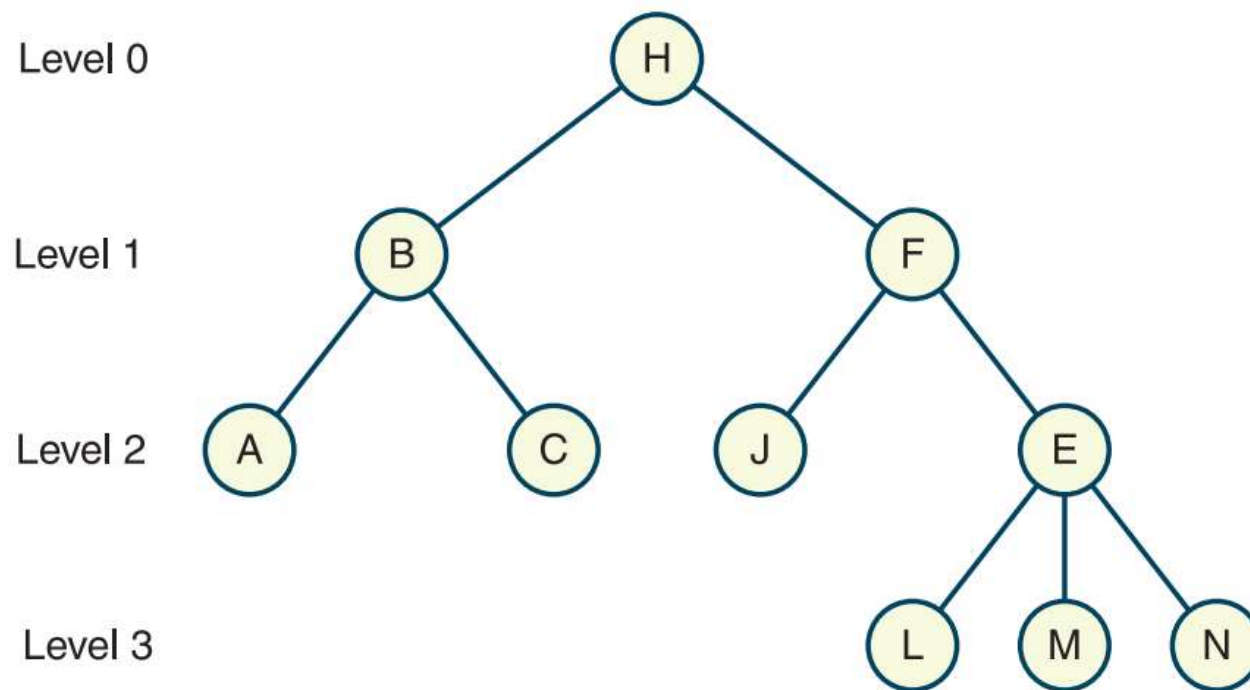
An overview of trees

- ❑ In the linear data structures, all items except for the first have a distinct predecessor, and all items except the last have a distinct successor.
- ❑ In a tree, the ideas of predecessor and successor are replaced with those of **parent** and **child**. Trees have two main characteristics:
 - Each item can have multiple children.
 - All items, except an item called the **root**, have exactly one parent. The root has no parent.



Term	Definition
Node	An item stored in a tree.
Root	The topmost node in a tree. It is the only node without a parent.
Child	A node immediately below and directly connected to a given node. A node can have more than one child, and its children are viewed as organized in left-to-right order. The leftmost child is called the first child, and the rightmost is called the last child.
Parent	A node immediately above and directly connected to a given node. A node can have only one parent.
Siblings	The children of a common parent.
Leaf	A node that has no children.
Interior node	A node that has at least one child.

Term	Definition
Edge/Branch/Link	The line that connects a parent to its child.
Descendant	A node's children, its children's children, and so on, down to the leaves.
Ancestor	A node's parent, its parent's parent, and so on, up to the root.
Path	The sequence of edges that connect a node and one of its descendants.
Path length	The number of edges in a path.
Depth or level	The depth or level of a node equals the length of the path connecting it to the root. Thus, the root depth or level of the root is 0. Its children are at level 1, and so on.
Height	The length of the longest path in the tree; put differently, the maximum level number among leaves in the tree.
Subtree	The tree formed by considering a node and all its descendants.



PROPERTY	VALUE
Number of nodes	10
Height	3
Root node	H
Leaves	A, C, J, L, M, N
Interior nodes	H, B, F, E
Nodes at level 2	A, C, J, E
Ancestors of E	F, H
Descendants of F	J, E, L, M, N
Node in the rightmost subtree of F	E, L, M, N

Figure 10-1 Some properties of a tree

General Trees and Binary Trees

- ❑ The tree shown in Figure 10-1 is sometimes called a **general tree** to distinguish it from a special category called a **binary tree**.
- ❑ In a binary tree, each node has at most two children, referred to as the **left child** and the **right child**

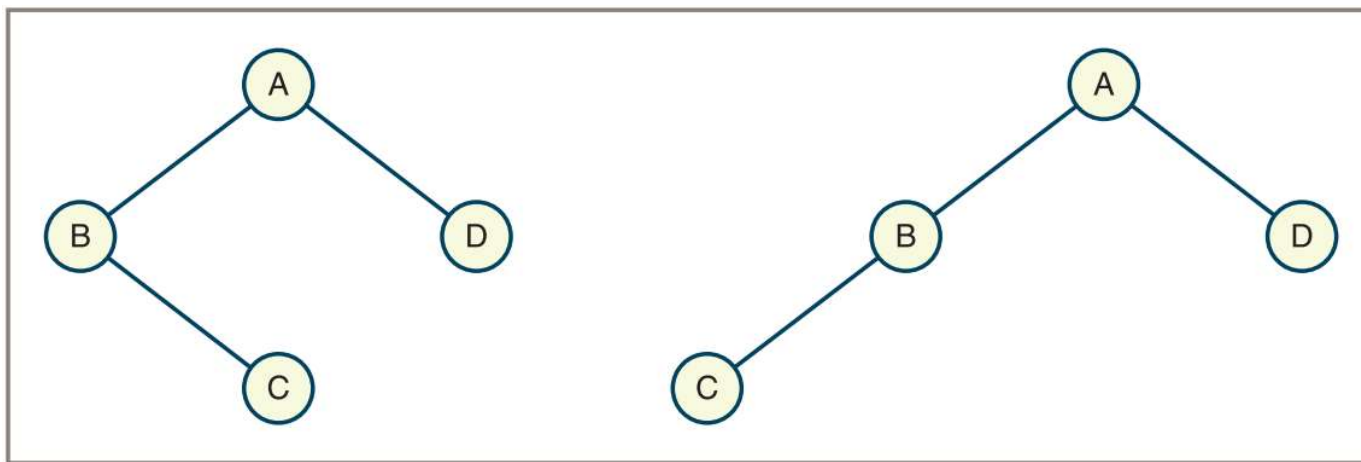


Figure 10-2 Two unequal binary trees that have the same sets of nodes

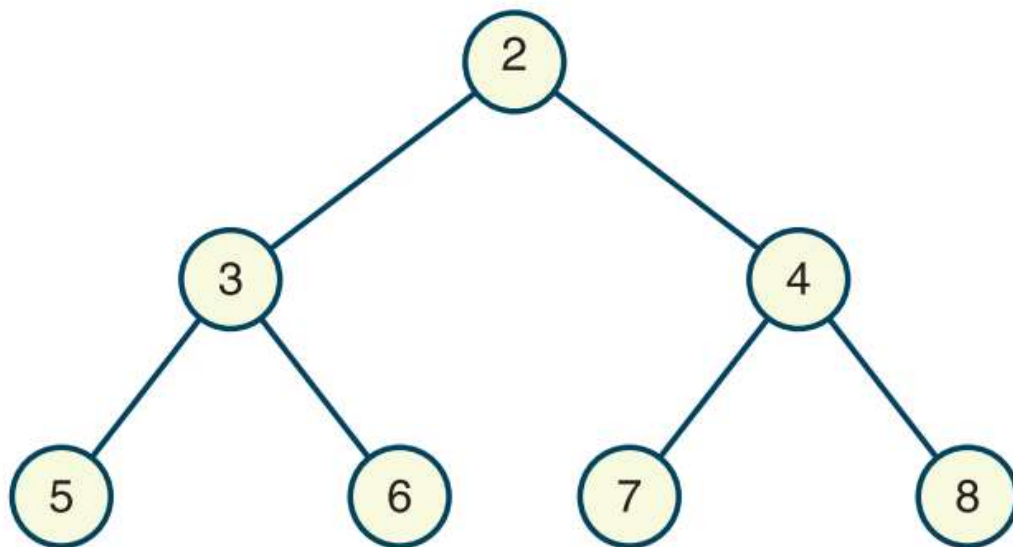


Recursive Definitions of Tree

- ❑ **General tree**—A general tree is either empty or consists of a finite set of nodes T . One node r is distinguished from all others and is called the root. In addition, the set $T - \{r\}$ is partitioned into disjointed subsets, each of which is a general tree.
- ❑ **Binary tree**—A binary tree is either empty or consists of a root plus a left subtree and a right subtree, each of which is a binary tree.

Exercises

Use the following tree to answer the next six questions.



1. What are the leaf nodes in the tree?
2. What are the interior nodes in the tree?
3. What are the siblings of node 7?
4. What is the height of the tree?
5. How many nodes are in level 2?
6. Is the tree a general tree, a binary tree, or both?



Some applications

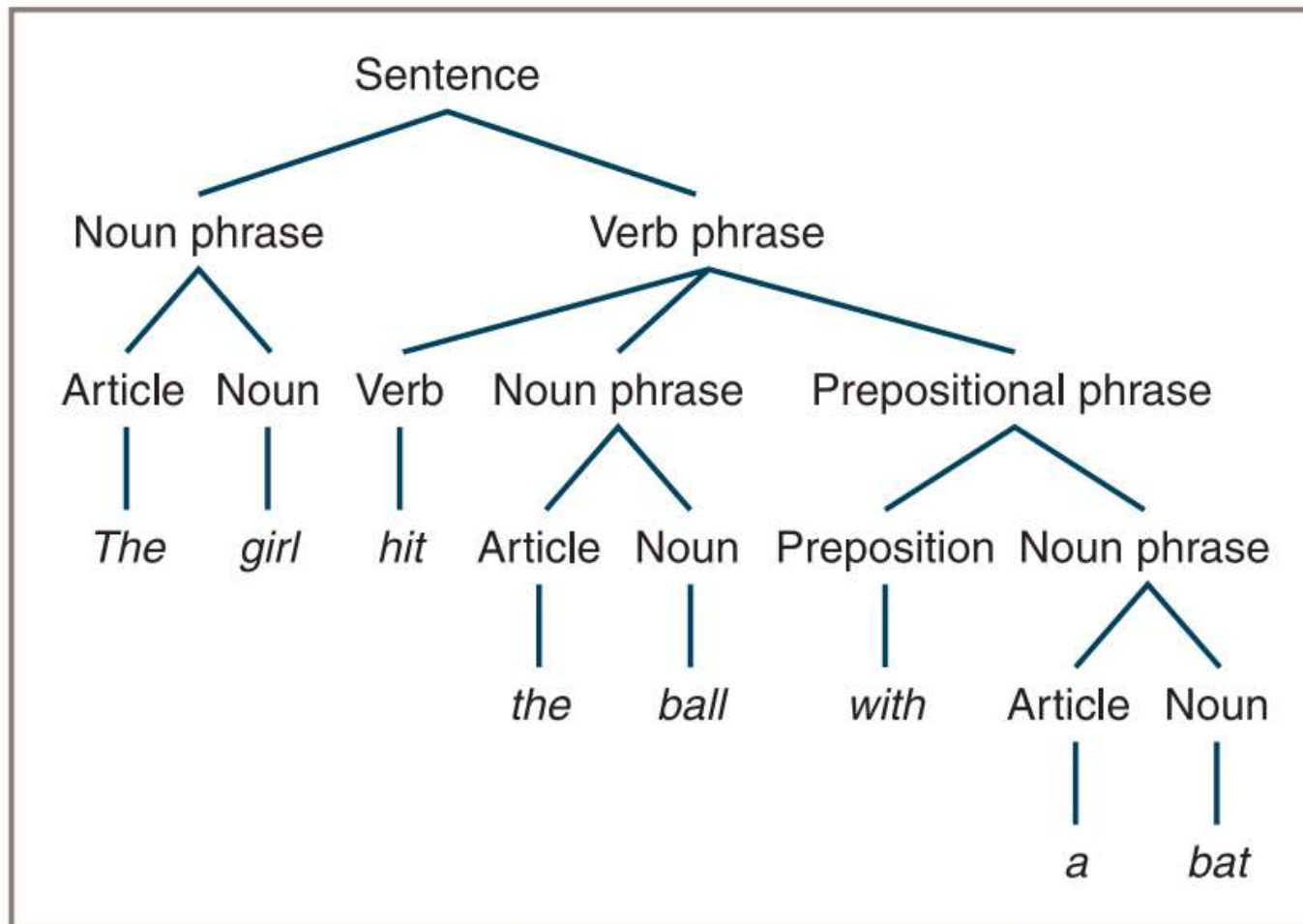


Figure 10-3 A parse tree for a sentence

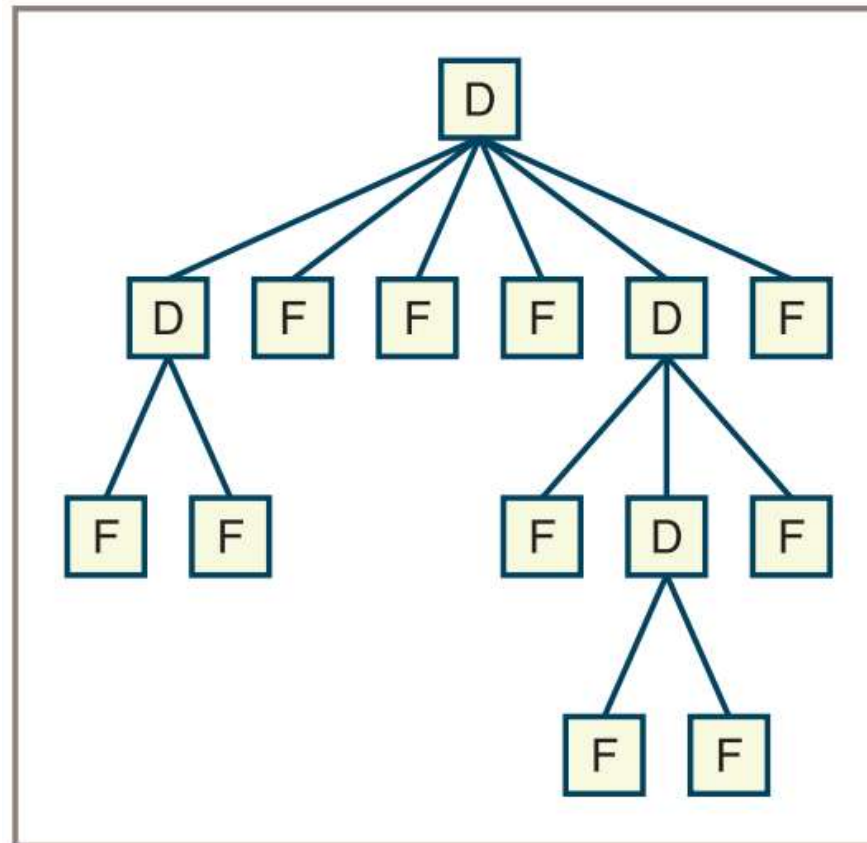


Figure 10-4 A file system structure

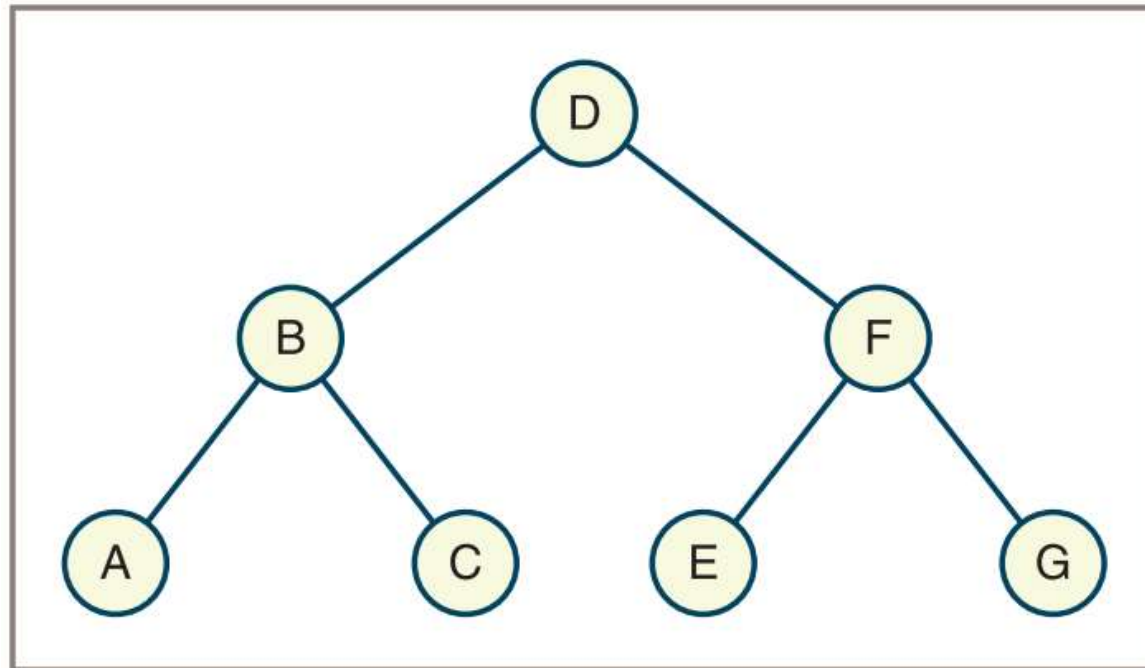


Figure 10-5 A sorted collection as a binary search tree



8.1. General Trees



8.1.2 The Tree Abstract Data Type

□ A position object for a tree supports the method:

➤ `p.element()`: Return the element stored at position `p`.

□ *accessor methods*, allowing a user to navigate the various positions of a tree:

T.root(): Return the position of the root of tree T,
or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p,
or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that
are contained in tree T.

T.is_empty(): Return True if tree T does not contain any positions.

T.positions(): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

```

1 class Tree:
2     """ Abstract base class representing a tree structure."""
3
4     #----- nested Position class -----
5     class Position:
6         """ An abstraction representing the location of a single element."""
7
8         def element(self):
9             """ Return the element stored at this Position."""
10            raise NotImplementedError('must be implemented by subclass')
11
12        def __eq__(self, other):
13            """ Return True if other Position represents the same location."""
14            raise NotImplementedError('must be implemented by subclass')
15
16        def __ne__(self, other):
17            """ Return True if other does not represent the same location."""
18            return not (self == other)                # opposite of __eq__

```

```

19
20 # ----- abstract methods that concrete subclass must support -----
21 def root(self):
22     """ Return Position representing the tree's root (or None if empty). """
23     raise NotImplementedError('must be implemented by subclass')
24
25 def parent(self, p):
26     """ Return Position representing p's parent (or None if p is root). """
27     raise NotImplementedError('must be implemented by subclass')
28
29 def num_children(self, p):
30     """ Return the number of children that Position p has. """
31     raise NotImplementedError('must be implemented by subclass')
32
33 def children(self, p):
34     """ Generate an iteration of Positions representing p's children. """
35     raise NotImplementedError('must be implemented by subclass')

```



```
36
37 def __len__(self):
38     """Return the total number of elements in the tree."""
39     raise NotImplementedError('must be implemented by subclass')
```

Code Fragment 8.1: A portion of our Tree abstract base class (continued in Code Fragment 8.2).

----- concrete methods implemented in this class -----

```
def is_root(self, p):
```

```
    """ Return True if Position p represents the root of the tree. """
```

```
    return self.root( ) == p
```

```
def is_leaf(self, p):
```

```
    """ Return True if Position p does not have any children. """
```

```
    return self.num_children(p) == 0
```

```
def is_empty(self):
```

```
    """ Return True if the tree is empty. """
```

```
    return len(self) == 0
```

Code Fragment 8.2: Some concrete methods of our Tree abstract base class.



8.1.3 Computing Depth and Height



8.1.3 Computing Depth

- Let p be the position of a node of a tree T . The *depth* of p is the number of ancestors of p , excluding p itself.
- The depth of p can also be recursively defined as follows:
 - If p is the root, then the depth of p is 0.
 - Otherwise, the depth of p is one plus the depth of the parent of p



```
52 def depth(self, p):  
53     """Return the number of levels separating Position p from the root."""  
54     if self.is_root(p):  
55         return 0  
56     else:  
57         return 1 + self.depth(self.parent(p))
```

Code Fragment 8.3: Method depth of the Tree class.



8.1.3 Computing Height

- The **height** of a position p in a tree T is also defined recursively:
 - If p is a leaf, then the height of p is 0.
 - Otherwise, the height of p is one more than the maximum of the heights of p 's children.
- The **height** of a nonempty tree T is the height of the root of T .
- The height of a nonempty tree T is equal to the maximum of the depths of its leaf positions



```
58  def _height1(self):                                # works, but  $O(n^2)$  worst-case time
59      """Return the height of the tree."""
60      return max(self.depth(p) for p in self.positions() if self.is_leaf(p))
```

Code Fragment 8.4: Method `_height1` of the `Tree` class. Note that this method calls the `depth` method.

- Complexity: $O(n^2)$ where n is the number of nodes of the subtree with root p .



```
61  def _height2(self, p):                                # time is linear in size of subtree
62      """Return the height of the subtree rooted at Position p."""
63      if self.is_leaf(p):
64          return 0
65      else:
66          return 1 + max(self._height2(c) for c in self.children(p))
```

Code Fragment 8.5: Method `_height2` for computing the height of a subtree rooted at a position p of a Tree.

□ Complexity: $O(n)$ where n is the number of nodes of the subtree with root p .



```
67  def height(self, p=None):
68      """ Return the height of the subtree rooted at Position p.
69
70      If p is None, return the height of the entire tree.
71      """
72      if p is None:
73          p = self.root()
74      return self._height2(p)           # start _height2 recursion
```

Code Fragment 8.6: Public method `Tree.height` that computes the height of the entire tree by default, or a subtree rooted at given position, if specified.



Binary trees

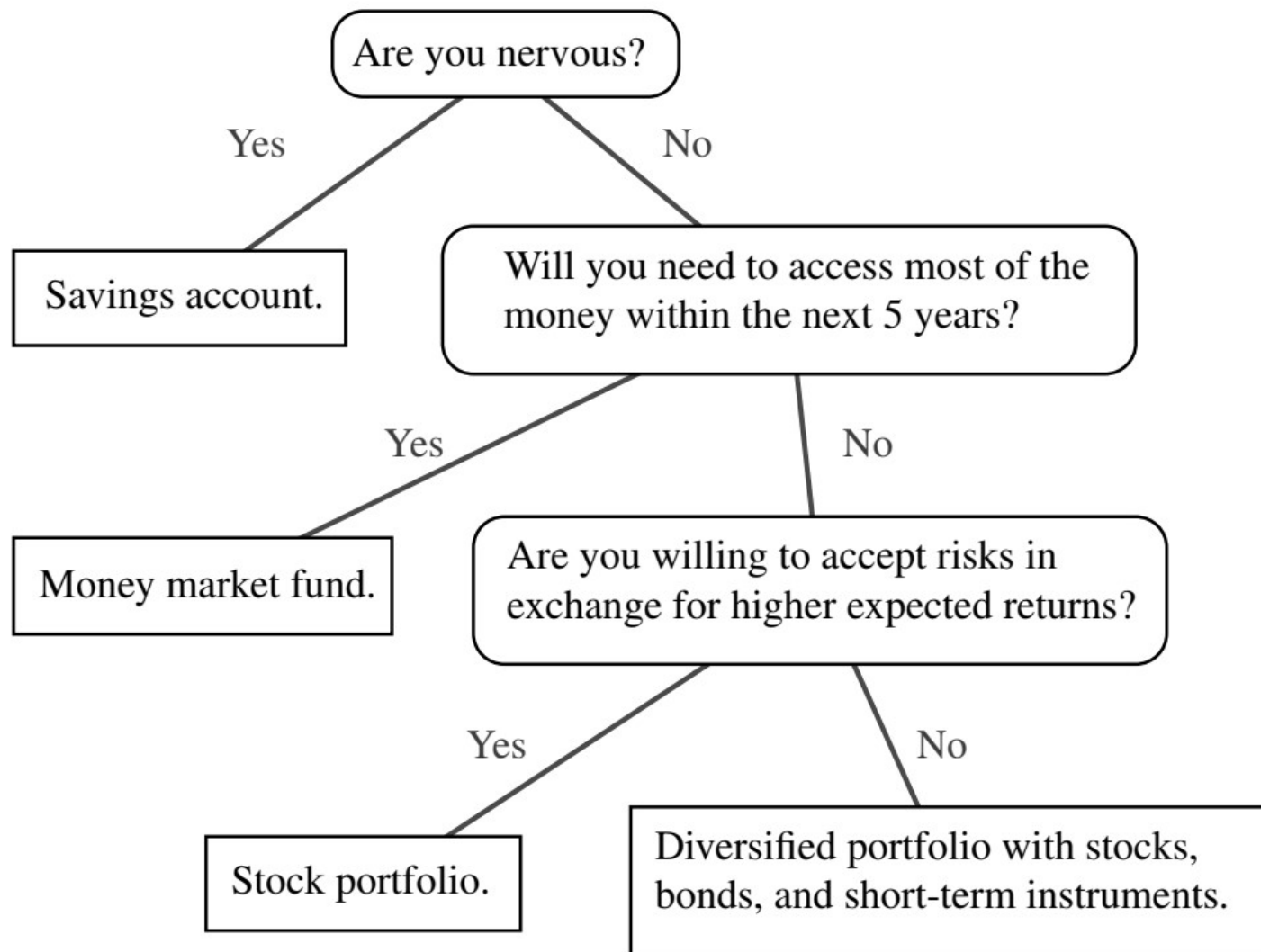


Figure 8.7: A decision tree providing investment advice.

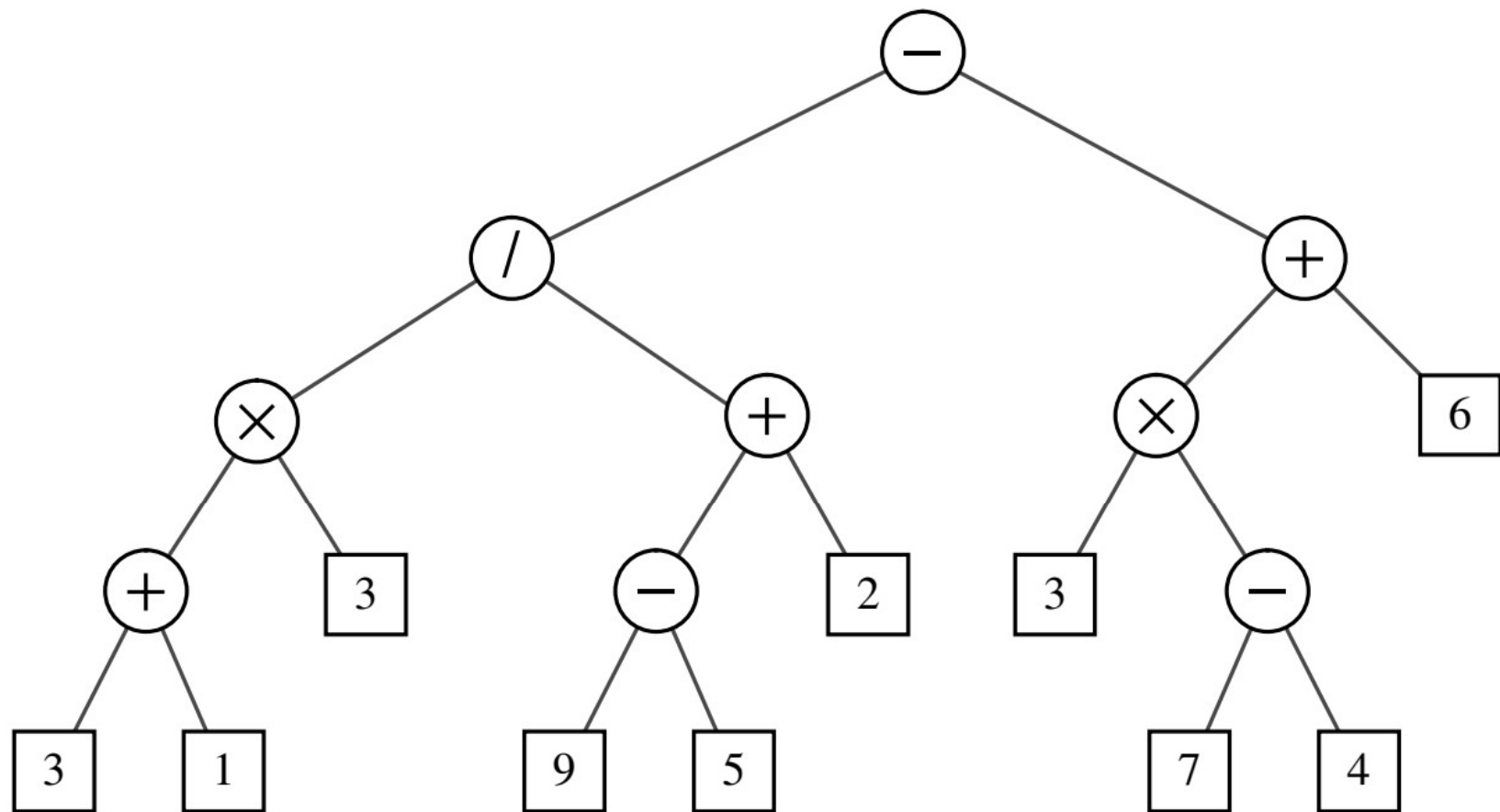


Figure 8.8: A binary tree representing an arithmetic expression. This tree represents the expression $((((3 + 1) \times 3) / ((9 - 5) + 2)) - ((3 \times (7 - 4)) + 6))$. The value associated with the internal node labeled “/” is 2.



A Recursive Binary Tree Definition

- ❑ A node r , called the root of T , that stores an element
- ❑ A binary tree (possibly empty), called the left subtree of T
- ❑ A binary tree (possibly empty), called the right subtree of T



8.2.1 The Binary Tree Abstract Data Type



As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

T.left(p): Return the position that represents the left child of p , or None if p has no left child.

T.right(p): Return the position that represents the right child of p , or None if p has no right child.

T.sibling(p): Return the position that represents the sibling of p , or None if p has no sibling.



The BinaryTree Abstract Base Class in Python

```

1 class BinaryTree(Tree):
2     """ Abstract base class representing a binary tree structure."""
3
4     # ----- additional abstract methods -----
5     def left(self, p):
6         """ Return a Position representing p's left child.
7
8         Return None if p does not have a left child.
9         """
10        raise NotImplementedError('must be implemented by subclass')
11
12    def right(self, p):
13        """ Return a Position representing p's right child.
14
15        Return None if p does not have a right child.
16        """
17        raise NotImplementedError('must be implemented by subclass')

```

```

18
19 # ----- concrete methods implemented in this class -----
20 def sibling(self, p):
21     """Return a Position representing p's sibling (or None if no sibling)."""
22     parent = self.parent(p)
23     if parent is None:                # p must be the root
24         return None                  # root has no sibling
25     else:
26         if p == self.left(parent):
27             return self.right(parent)    # possibly None
28         else:
29             return self.left(parent)     # possibly None
30
31 def children(self, p):
32     """Generate an iteration of Positions representing p's children."""
33     if self.left(p) is not None:
34         yield self.left(p)
35     if self.right(p) is not None:
36         yield self.right(p)

```

Code Fragment 8.7: A BinaryTree abstract base class that extends the existing Tree



8.2.2 Properties of Binary Trees

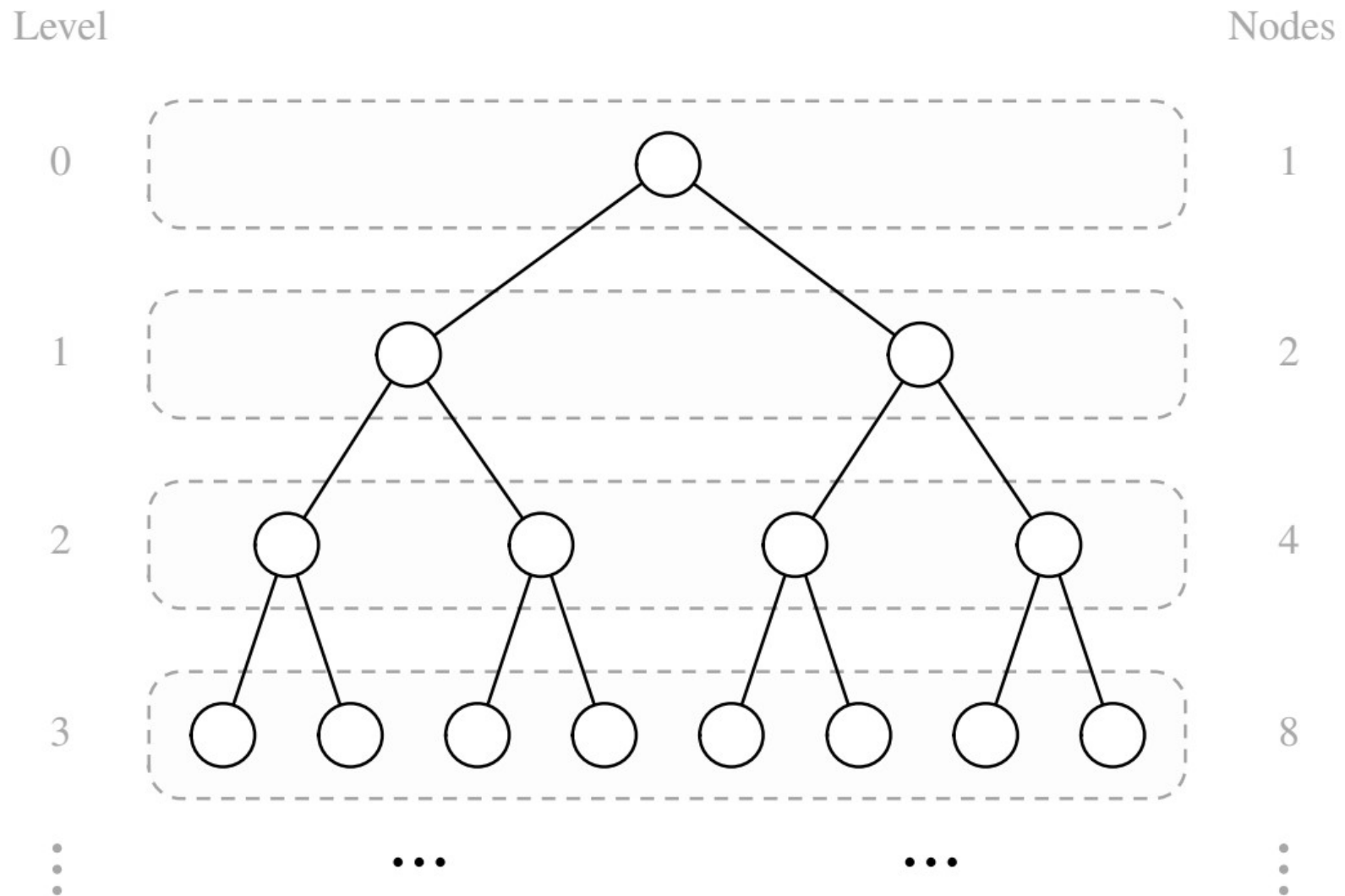


Figure 8.9: Maximum number of nodes in the levels of a binary tree.

Full binary tree



- Contains the maximum number of nodes for a given height H

Height of the Tree	Number of Nodes in the Tree
0	1
1	3
2	7
3	15

Table 10-2 The Relationship Between the Height and the Number of Nodes in Full Binary Tree

Proposition 8.8: *Let T be a nonempty binary tree, and let n , n_E , n_I and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T , respectively. Then T has the following properties:*

1. $h + 1 \leq n \leq 2^{h+1} - 1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq n - 1$

Also, if T is proper, then T has the following properties:

1. $2h + 1 \leq n \leq 2^{h+1} - 1$
2. $h + 1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq (n - 1)/2$



Proposition 8.9: *In a nonempty proper binary tree T , with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.*



Height of full binary tree given its node

Let's generalize from this table. What is the number of nodes, N , contained in a full binary tree of height H ? To express N in terms of H , you start with the root (1 node), add its children (2 nodes), add their children (4 nodes), and so on, as follows:

$$\begin{aligned} N &= 1 + 2 + 4 + \dots + 2^H \\ &= 2^{H+1} - 1 \end{aligned}$$

And what is the height, H , of a full binary tree with N nodes? Using simple algebra, you get

$$H = \log_2(N + 1) - 1$$



Binary tree

- ❑ A full binary tree: level k has exactly $2^k - 1$ nodes
- ❑ A **perfectly balanced binary tree**, which includes a complete complement of nodes at each level but the last one, is bushy enough to support worst-case logarithmic access to leaf nodes.
- ❑ A **complete binary tree**, in which any nodes on the last level are filled in from left to right, is, like a full binary tree, a special case of a perfectly balanced binary tree.

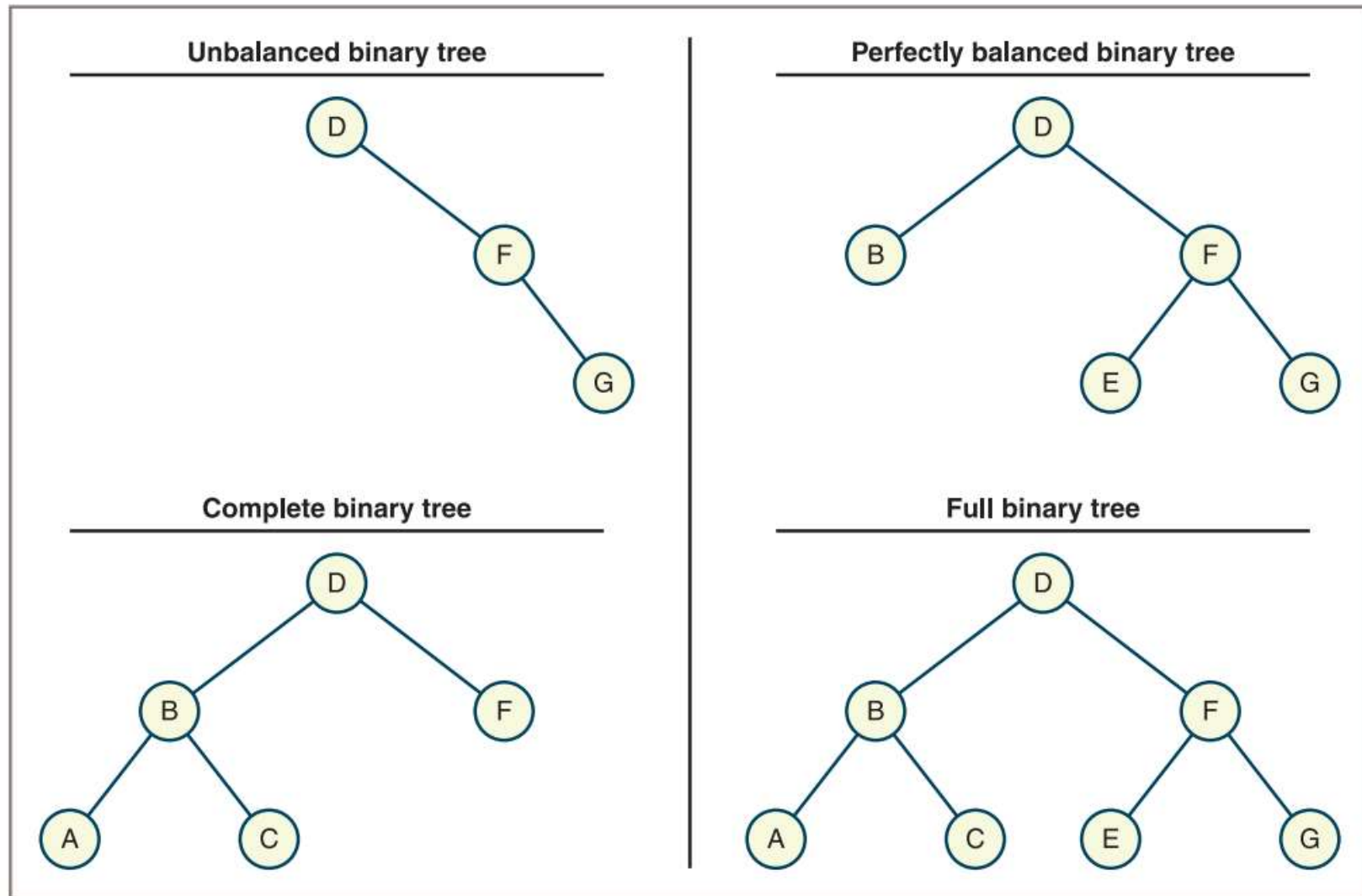


Figure 10-7 Four types of shapes of binary trees



Exercises

1. What is the difference between a perfectly balanced binary tree and a complete binary tree?
 2. What is the difference between a complete binary tree and a full binary tree?
 3. A full binary tree has a height of 5. How many nodes does it contain?
 4. A complete binary tree contains 125 nodes. What is its height?
 5. How many nodes are on a given level L in a full binary tree? Express your answer in terms of L .
-



8.3 Implementing Trees

8.3.1 Linked Structure for Binary Trees

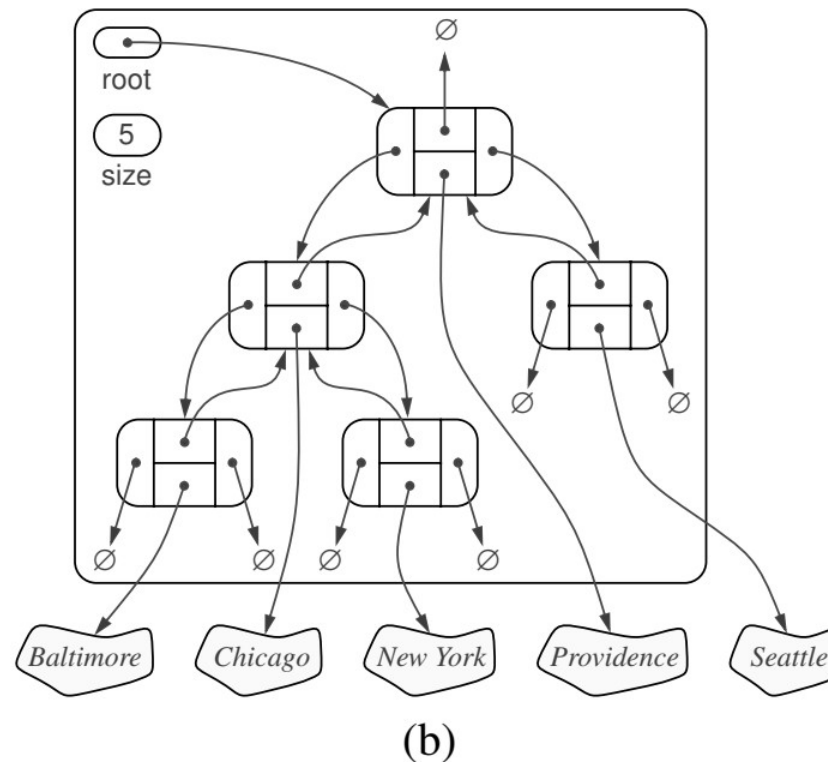
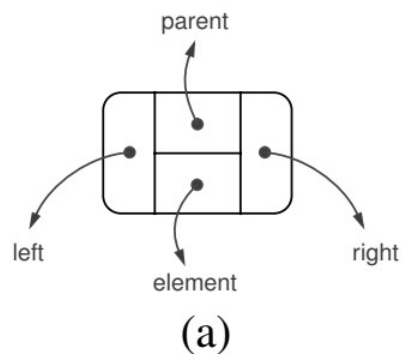


Figure 8.11: A linked structure for representing: (a) a single node; (b) a binary tree.



Python Implementation of a Linked Binary Tree Structure

Operations for Updating a Linked Binary Tree



- T.add_root(e):** Create a root for an empty tree, storing e as the element, and return the position of that root; an error occurs if the tree is not empty.
- T.add_left(p, e):** Create a new node storing element e, link the node as the left child of position p, and return the resulting position; an error occurs if p already has a left child.
- T.add_right(p, e):** Create a new node storing element e, link the node as the right child of position p, and return the resulting position; an error occurs if p already has a right child.
- T.replace(p, e):** Replace the element stored at position p with element e, and return the previously stored element.
- T.delete(p):** Remove the node at position p, replacing it with its child, if any, and return the element that had been stored at p; an error occurs if p has two children.
- T.attach(p, T1, T2):** Attach the internal structure of trees T1 and T2, respectively, as the left and right subtrees of leaf position p of T, and reset T1 and T2 to empty trees; an error condition occurs if p is not a leaf.

```

1  class LinkedBinaryTree(BinaryTree):
2      """Linked representation of a binary tree structure."""
3
4      class _Node:          # Lightweight, nonpublic class for storing a node.
5          __slots__ = '_element', '_parent', '_left', '_right'
6          def __init__(self, element, parent=None, left=None, right=None):
7              self._element = element
8              self._parent = parent
9              self._left = left
10             self._right = right
11
12     class Position(BinaryTree.Position):
13         """An abstraction representing the location of a single element."""
14
15         def __init__(self, container, node):
16             """Constructor should not be invoked by user."""
17             self._container = container
18             self._node = node

```

```
19
20 def element(self):
21     """ Return the element stored at this Position."""
22     return self._node._element
23
24 def __eq__(self, other):
25     """ Return True if other is a Position representing the same location."""
26     return type(other) is type(self) and other._node is self._node
27
28 def _validate(self, p):
29     """ Return associated node, if position is valid."""
30     if not isinstance(p, self.Position):
31         raise TypeError('p must be proper Position type')
32     if p._container is not self:
33         raise ValueError('p does not belong to this container')
34     if p._node._parent is p._node:      # convention for deprecated nodes
35         raise ValueError('p is no longer valid')
36     return p._node
```



```
37  
38 def _make_position(self, node):
```

```
39     """ Return Position instance for given node (or None if no node). """
```

```
40     return self.Position(self, node) if node is not None else None
```

Code Fragment 8.8: The beginning of our `LinkedBinaryTree` class (continued in Code Fragments 8.9 through 8.11).


```
41  #----- binary tree constructor -----
42  def __init__(self):
43      """ Create an initially empty binary tree. """
44      self._root = None
45      self._size = 0
46
47  #----- public accessors -----
48  def __len__(self):
49      """ Return the total number of elements in the tree. """
50      return self._size
51
52  def root(self):
53      """ Return the root Position of the tree (or None if tree is empty). """
54      return self._make_position(self._root)
```

```
56 def parent(self, p):
57     """ Return the Position of p's parent (or None if p is root). """
58     node = self._validate(p)
59     return self._make_position(node._parent)
60
61 def left(self, p):
62     """ Return the Position of p's left child (or None if no left child). """
63     node = self._validate(p)
64     return self._make_position(node._left)
65
66 def right(self, p):
67     """ Return the Position of p's right child (or None if no right child). """
68     node = self._validate(p)
69     return self._make_position(node._right)
70
```



```
71 def num_children(self, p):  
72     """ Return the number of children of Position p. """  
73     node = self._validate(p)  
74     count = 0  
75     if node._left is not None:           # left child exists  
76         count += 1  
77     if node._right is not None:         # right child exists  
78         count += 1  
79     return count
```

Code Fragment 8.9: Public accessors for our `LinkedBinaryTree` class. The class begins in Code Fragment 8.8 and continues in Code Fragments 8.10 and 8.11.



```
80 def _add_root(self, e):
81     """ Place element e at the root of an empty tree and return new Position
82
83     Raise ValueError if tree nonempty.
84     """
85     if self._root is not None: raise ValueError('Root exists')
86     self._size = 1
87     self._root = self._Node(e)
88     return self._make_position(self._root)
89
90 def _add_left(self, p, e):
91     """ Create a new left child for Position p, storing element e.
92
```

```

93     Return the Position of new node.
94     Raise ValueError if Position p is invalid or p already has a left child.
95     """
96     node = self._validate(p)
97     if node._left is not None: raise ValueError('Left child exists')
98     self._size += 1
99     node._left = self._Node(e, node)                # node is its parent
100    return self._make_position(node._left)
101
102    def _add_right(self, p, e):
103        """ Create a new right child for Position p, storing element e.
104
105        Return the Position of new node.
106        Raise ValueError if Position p is invalid or p already has a right child.
107        """
108        node = self._validate(p)
109        if node._right is not None: raise ValueError('Right child exists')
110        self._size += 1
111        node._right = self._Node(e, node)            # node is its parent
112        return self._make_position(node._right)

```



```
113
114 def _replace(self, p, e):
115     """ Replace the element at position p with e, and return old element. """
116     node = self._validate(p)
117     old = node._element
118     node._element = e
119     return old
```

Code Fragment 8.10: Nonpublic update methods for the `LinkedBinaryTree` class (continued in Code Fragment 8.11).

```

120 def _delete(self, p):
121     """Delete the node at Position p, and replace it with its child, if any.
122
123     Return the element that had been stored at Position p.
124     Raise ValueError if Position p is invalid or p has two children.
125     """
126     node = self._validate(p)
127     if self.num_children(p) == 2: raise ValueError('p has two children')
128     child = node._left if node._left else node._right      # might be None
129     if child is not None:
130         child._parent = node._parent      # child's grandparent becomes parent
131     if node is self._root:
132         self._root = child                # child becomes root
133     else:
134         parent = node._parent
135         if node is parent._left:
136             parent._left = child
137         else:
138             parent._right = child
139     self._size -= 1
140     node._parent = node                    # convention for deprecated node
141     return node._element
142

```



```

143 def _attach(self, p, t1, t2):
144     """ Attach trees t1 and t2 as left and right subtrees of external p."""
145     node = self._validate(p)
146     if not self.is_leaf(p): raise ValueError('position must be leaf')
147     if not type(self) is type(t1) is type(t2): # all 3 trees must be same type
148         raise TypeError('Tree types must match')
149     self._size += len(t1) + len(t2)
150     if not t1.is_empty(): # attached t1 as left subtree of node
151         t1._root._parent = node
152         node._left = t1._root
153         t1._root = None # set t1 instance to empty
154         t1._size = 0
155     if not t2.is_empty(): # attached t2 as right subtree of node
156         t2._root._parent = node
157         node._right = t2._root
158         t2._root = None # set t2 instance to empty
159         t2._size = 0

```

Code Fragment 8.11: Nonpublic update methods for the `LinkedBinaryTree` class (continued from Code Fragment 8.10).



Operation	Running Time
len, is_empty	$O(1)$
root, parent, left, right, sibling, children, num_children	$O(1)$
is_root, is_leaf	$O(1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$
add_root, add_left, add_right, replace, delete, attach	$O(1)$

Table 8.1: Running times for the methods of an n -node binary tree implemented with a linked structure. The space usage is $O(n)$.



8.3.3 Linked Structure for General Trees

Operation	Running Time
len, is_empty	$O(1)$
root, parent, is_root, is_leaf	$O(1)$
children(p)	$O(c_p + 1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$

Table 8.2: Running times of the accessor methods of an n -node general tree implemented with a linked structure. We let c_p denote the number of children of a position p . The space usage is $O(n)$.

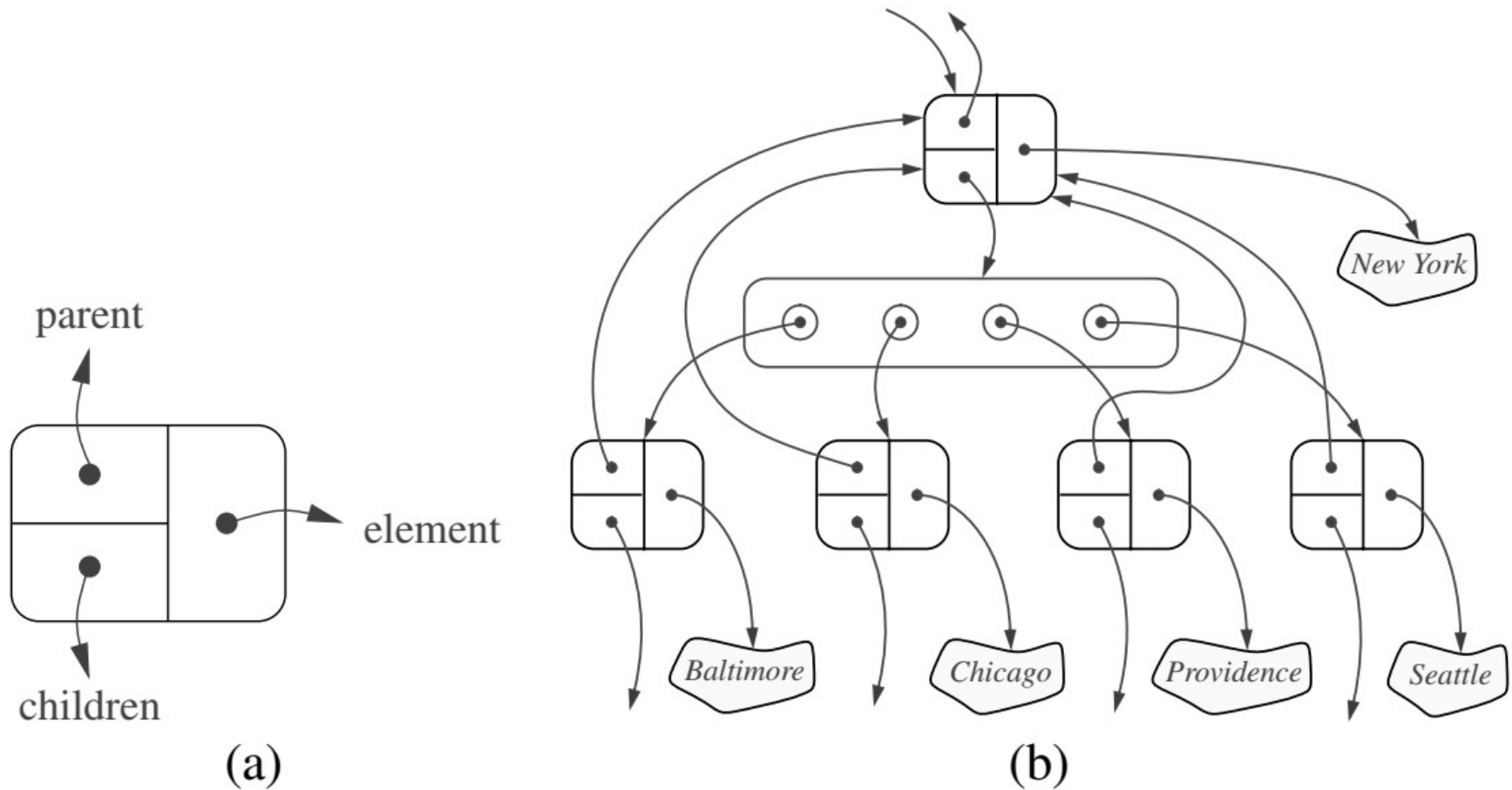


Figure 8.14: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.



8.4 Tree Traversal Algorithms



8.4 Tree Traversal Algorithms

- A **traversal** of a tree T is a systematic way of accessing, or “visiting,” all the positions of T .
- **preorder traversal:** the root of T is visited first and then the subtrees rooted at its children are traversed recursively.
- **postorder traversal:** recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name “postorder”)



8.4.1 Preorder of General Trees

Algorithm preorder(T, p):

perform the “visit” action for position p

for each child c in $T.children(p)$ **do**

preorder(T, c) {recursively traverse the subtree rooted at c }

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree T .

Preorder Traversal

- ❑ The **preorder traversal** algorithm visits a tree's root node and then traverses the left subtree and the right subtree in a similar manner.

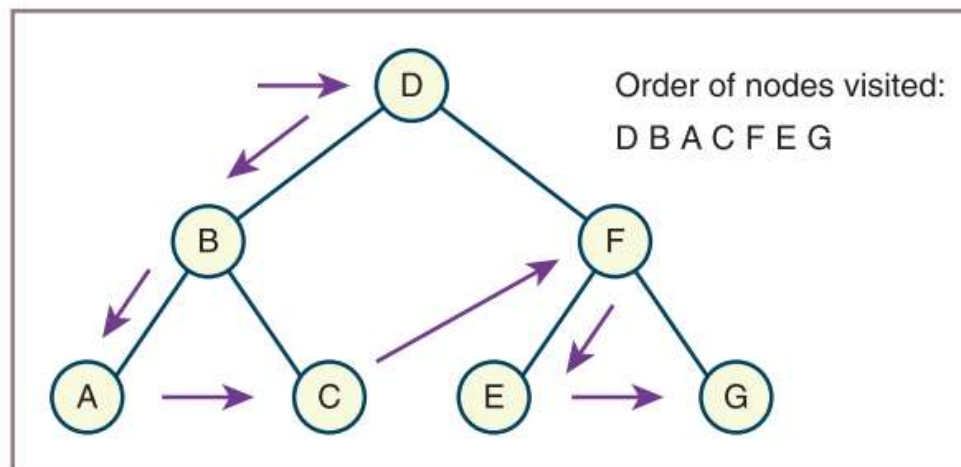


Figure 10-8 A preorder traversal

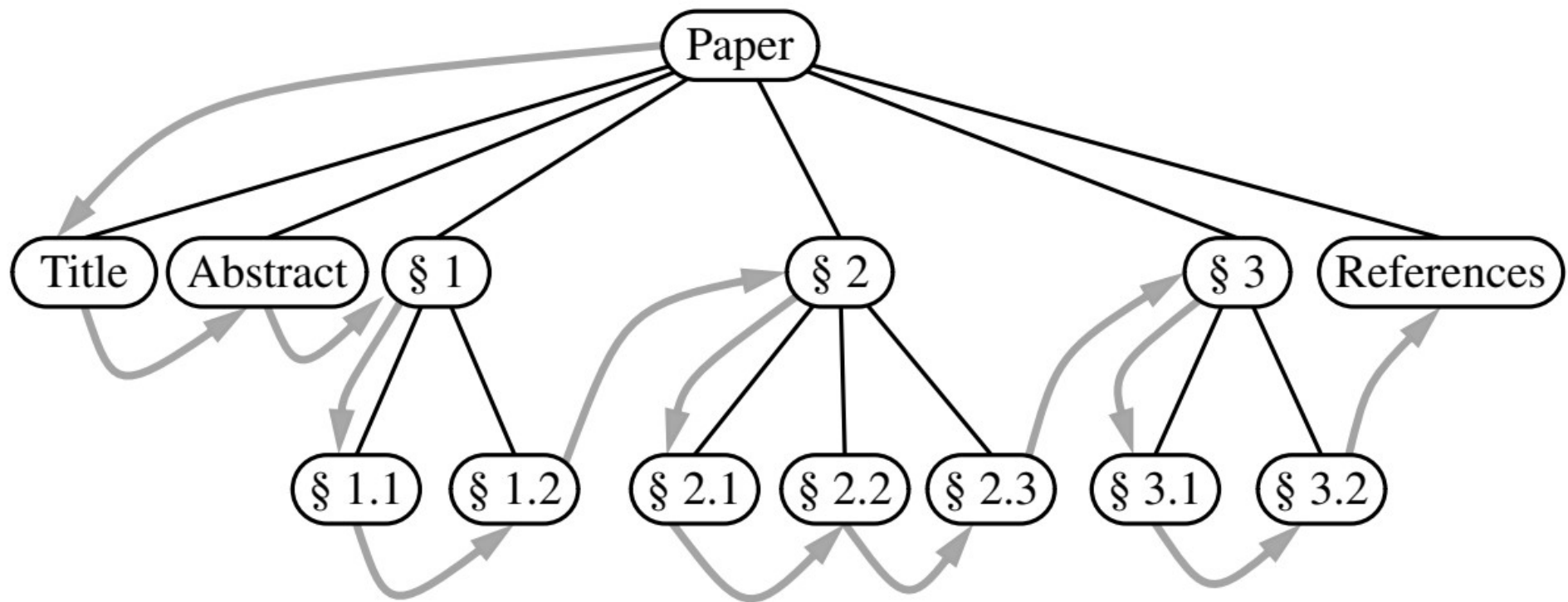


Figure 8.15: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.



8.4.1 Postorder of General Trees

Algorithm postorder(T , p):

```
for each child  $c$  in  $T.children(p)$  do
    postorder( $T$ ,  $c$ )           {recursively traverse the subtree rooted at  $c$ }
perform the “visit” action for position  $p$ 
```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree T .

Postorder Traversal

- The **postorder traversal** algorithm traverses the left subtree, traverses the right subtree, and visits the root node.

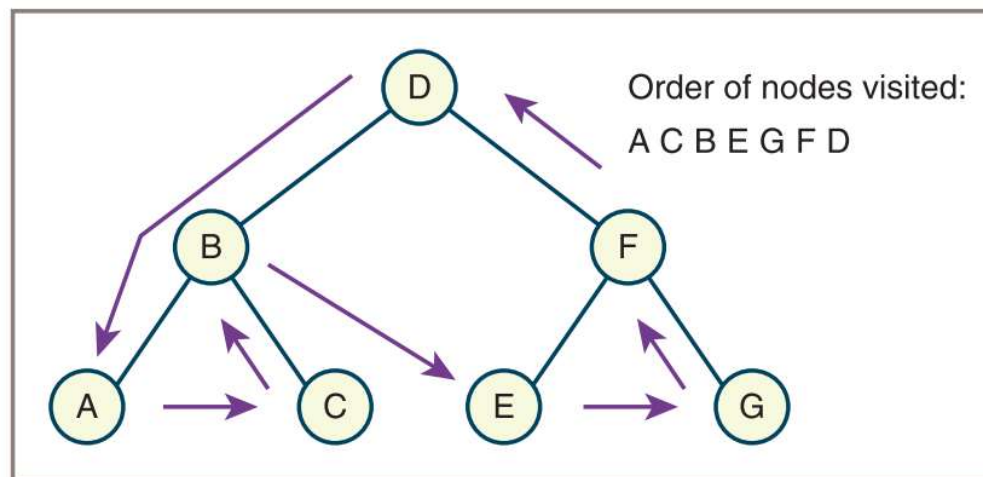


Figure 10-10 A postorder traversal

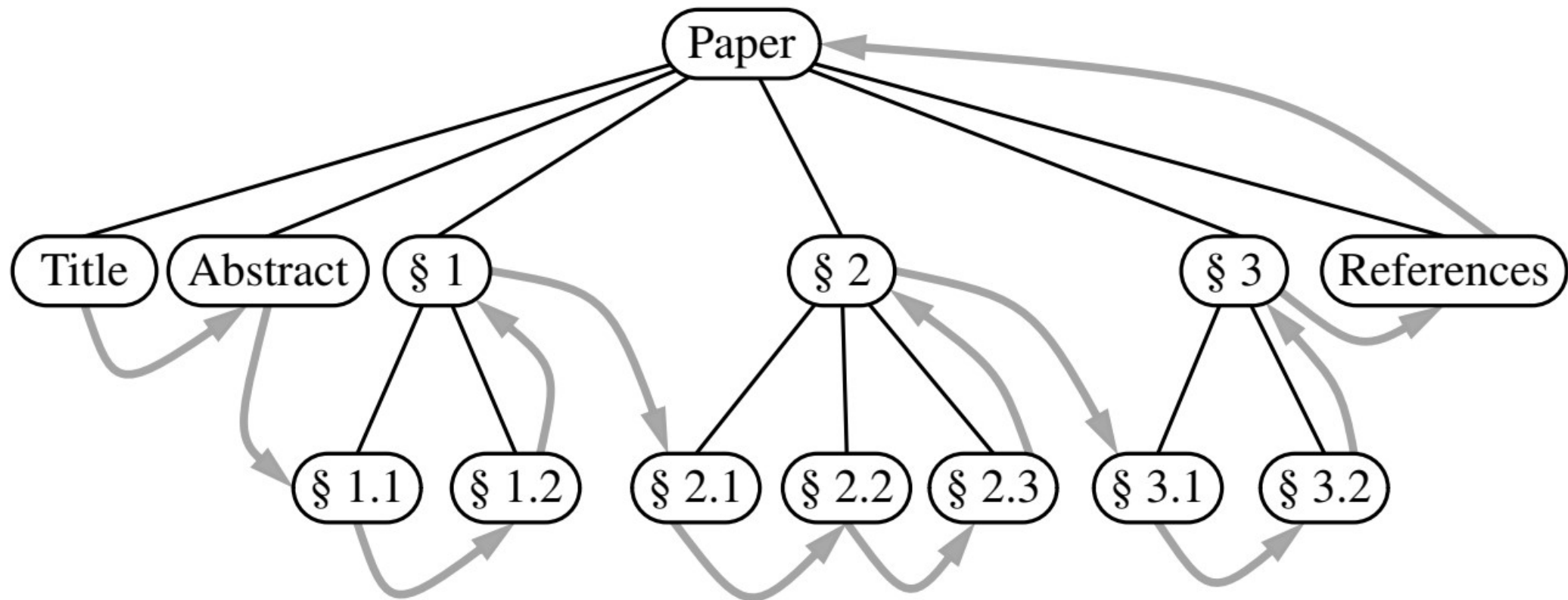


Figure 8.16: Postorder traversal of the ordered tree of Figure 8.15.

8.4.3 Inorder Traversal of a Binary Tree

□ ***inorder traversal***: we visit a position between the recursive traversals of its left and right subtrees.

Algorithm inorder(p):

if p has a left child lc **then**

inorder(lc)	{recursively traverse the left subtree of p}
-------------	--

perform the “visit” action for position p

if p has a right child rc **then**

inorder(rc)	{recursively traverse the right subtree of p}
-------------	---

Code Fragment 8.15: Algorithm inorder for performing an inorder traversal of a subtree rooted at position p of a binary tree.

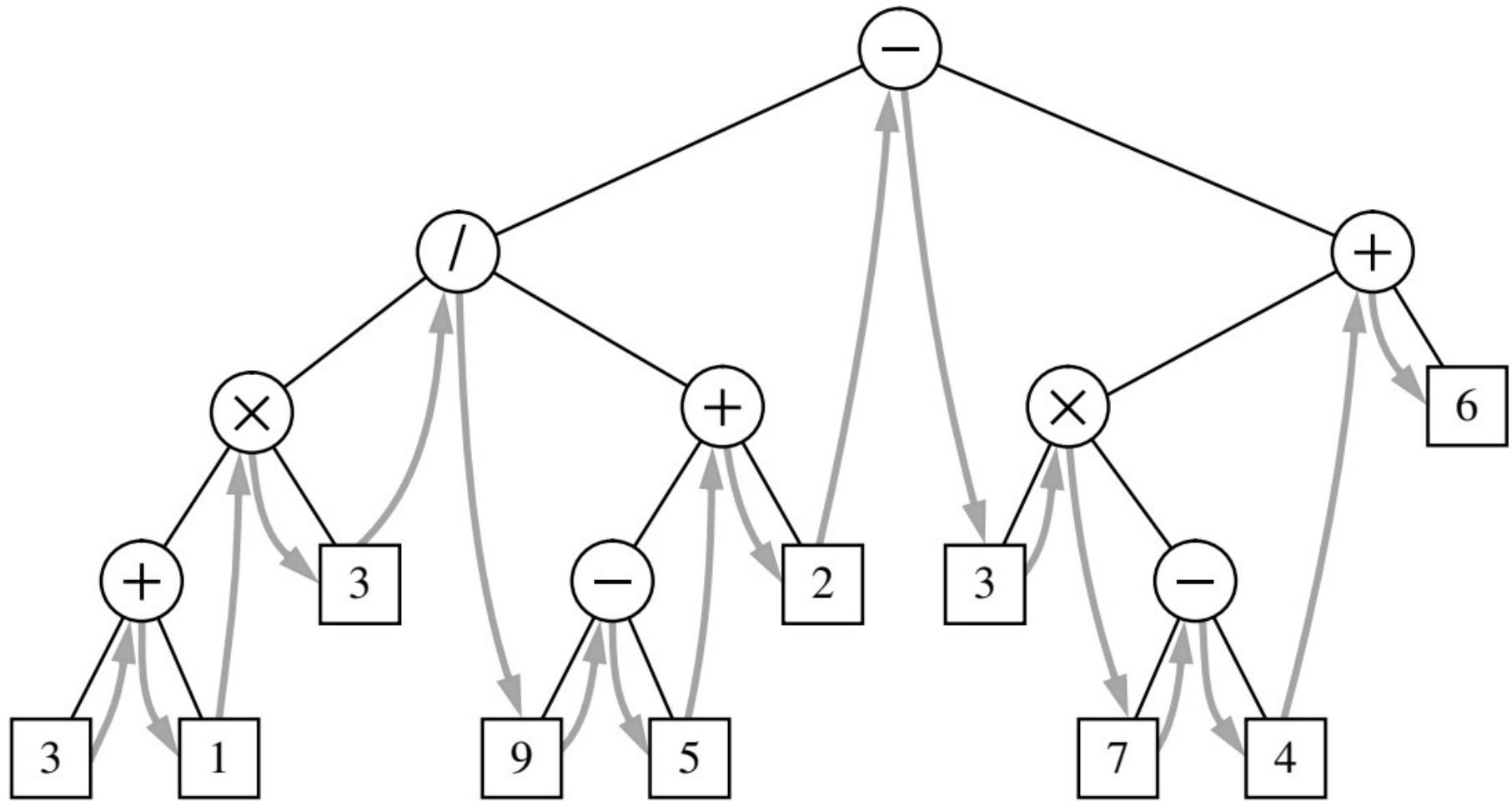


Figure 8.18: Inorder traversal of a binary tree.

8.4.2 Breadth-First Tree Traversal (Level order)

□ **breadth-first traversal:** traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d + 1$. (non-weighted shortest path application)

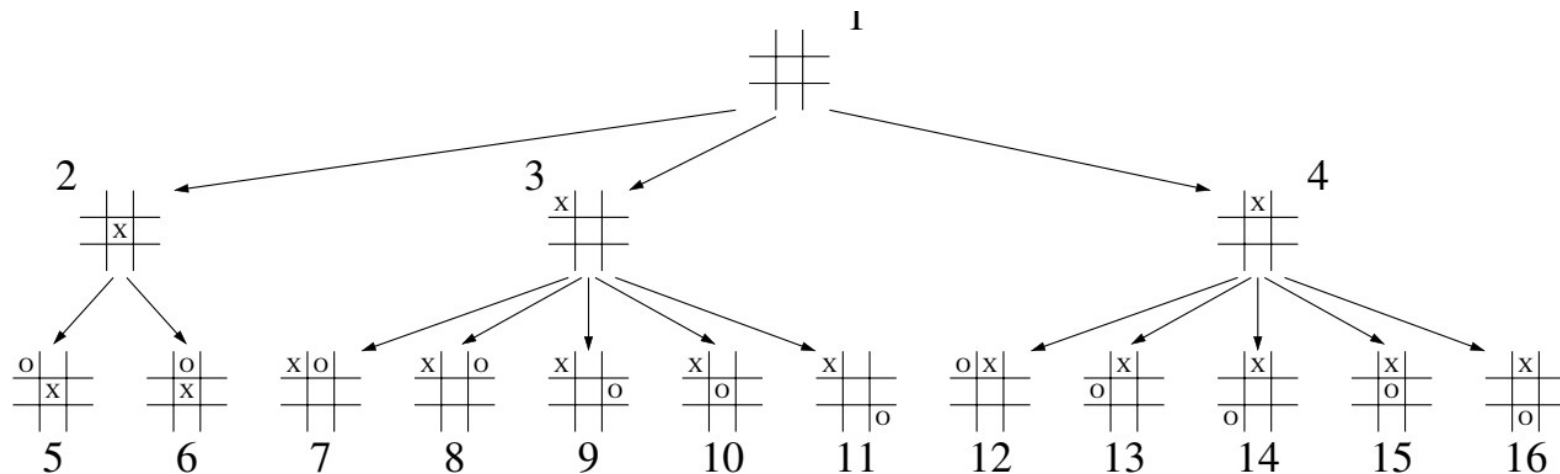


Figure 8.17: Partial game tree for Tic-Tac-Toe, with annotations displaying the order in which positions are visited in a breadth-first traversal.

Level order Traversal

- Beginning with level 0, the **level order traversal** algorithm visits the nodes at each level in left-to-right order

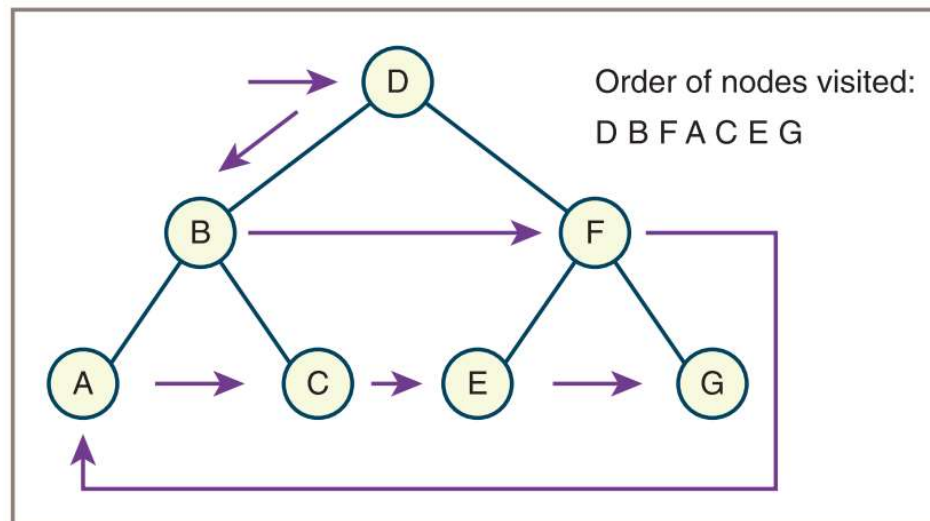


Figure 10-11 A level order traversal

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

Review Questions

1. The distinguished node at the beginning or top of a tree is called the:
 - a. Head node
 - b. Root node
 - c. Leaf node
2. A node without children is called a:
 - a. Single node
 - b. Leaf node
3. Each level k in a full binary tree contains:
 - a. $2k$ nodes
 - b. $2^k + 1$ nodes
 - c. $2^k - 1$ nodes
4. Assume that data are inserted into a binary search tree in the order D B A C F E G. A preorder traversal would return these data in the order:
 - a. D B A C F E G
 - b. A B C D E F G
5. Assume that data are inserted into a binary search tree in the order D B A C F E G. An inorder traversal would return these data in the order:
 - a. D B A C F E G
 - b. A B C D E F G

6. Assume that data are inserted into a binary search tree in the order A B C D E F G. The structure of this tree resembles that of a:
 - a. Full binary tree
 - b. List
7. The item removed from a min-heap is always the:
 - a. Smallest item
 - b. Largest item
8. A postorder traversal of an expression tree returns the expression in:
 - a. Infix form
 - b. Prefix form
 - c. Postfix form
9. The worst-case behavior of the search of a binary search tree is:
 - a. $O(\log n)$
 - b. $O(n)$
 - c. $O(n^2)$
10. Insertions and removals from a heap are:
 - a. Linear operations
 - b. Logarithmic operations